ĐẠI HỌC FPT

CHUYÊN NGÀNH AN TOÀN THÔNG TIN

FPT UNIVERSITY

BÁO CÁO DỰ ÁN

Môn học: Open Source Platform and Network Administration(OSP201)

Tên đề tài: WAF Log Monitor

Giảng viên hướng dẫn: **Mai Hoàng Đỉnh**

Lớp: **IA1904**

Sinh viên thực hiện:

– SE190227 Trần Xuân Trường

– SE193764 Mai Đức Huy

– SE192038 Trần Minh Phú

– SE193168 Lai Hộ Gia

Giới thiệu dự án

Tên dự án : WAF Log Monitor

- Lấy ý tưởng từ: Honeypot
- Cơ chế: Project này mô phỏng cách hệ thống web được bảo vệ. Khi attacker gửi request tới web, Nginx + ModSecurity (WAF) sẽ kiểm tra và chặn những request đáng ngờ. Tất cả log từ WAF và web server sẽ được gửi về Splunk để giám sát và phát hiện tấn công.
- Mục tiêu: giúp hiểu cơ chế WAF hoạt động và cách dùng Splunk để tập trung log, phát hiện các sự kiện bất thường
- Ý chính: Dùng Nginx + ModSecurity làm WAF, Splunk để thu và phân tích log → phát hiện tấn công web cơ bản

Mục tiêu của dự án

Hiểu cơ chế hoạt động của **WAF (ModSecurity + Nginx)**.

Xây dựng **môi trường mô phỏng tấn công web** và thu log thực tế.

Dùng **Splunk** để thu thập, phân tích, và phát hiện hành vi tấn công.

Rèn luyện kỹ năng **phân tích log, điều tra sự cố bảo mật**

Cơ sở lý thuyết

WAF (Web Application Firewall)

Là tường lửa ứng dụng web, hoạt động ở tầng 7 (Application Layer).

Phân tích nội dung request HTTP/HTTPS để chặn tấn công.

ModSecurity là module phổ biến của Nginx và Apache.

ModSecurity + OWASP CRS

ModSecurity hoạt động theo **rule** (quy tắc phát hiện).

OWASP CRS (Core Rule Set) chứa các rule nhận dạng tấn công phổ biến.

Các rule được viết theo pattern như SQL Injection, XSS, Command Injection, v.v.

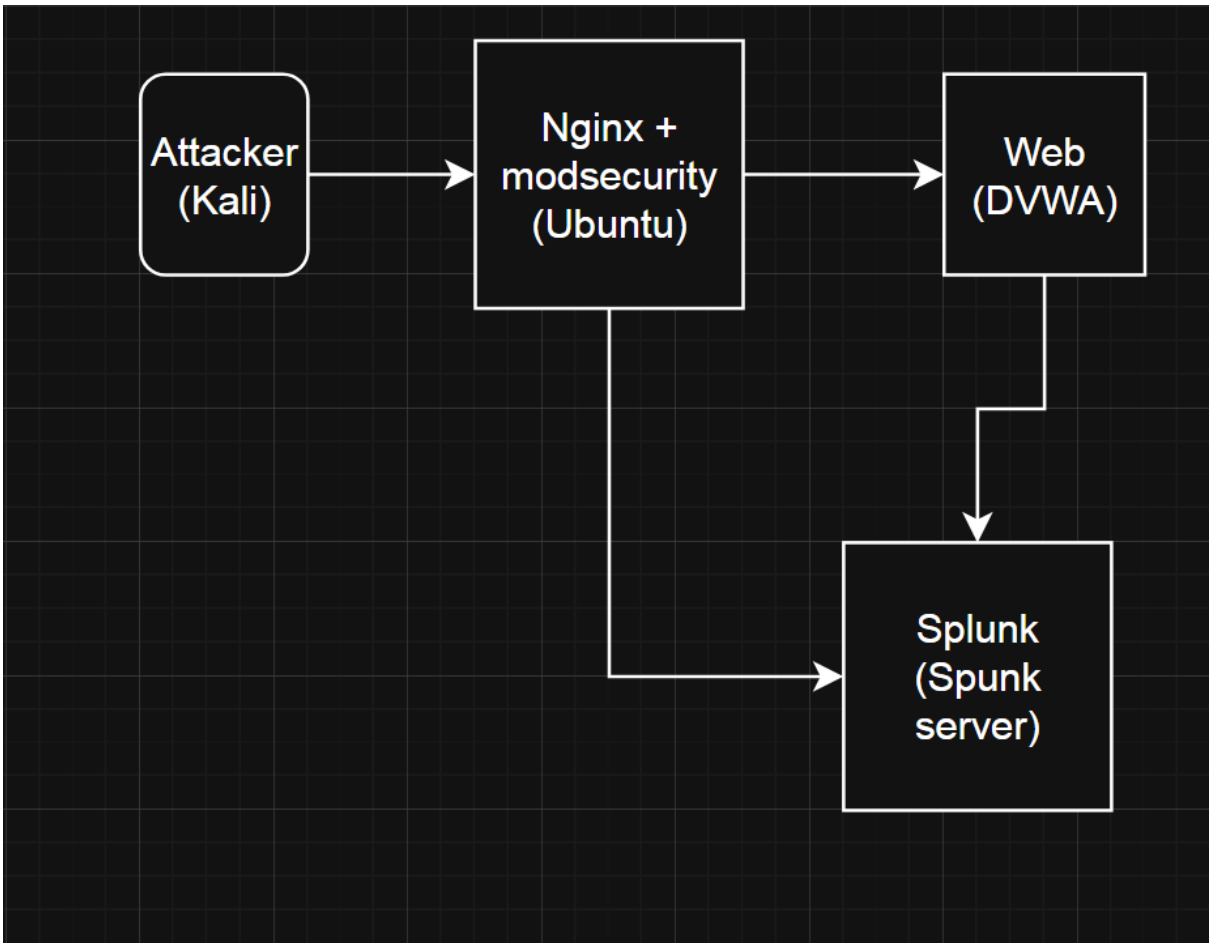
Splunk

Là **SIEM (Security Information and Event Management)** – hệ thống quản lý và phân tích log.

Có thể thu thập log từ nhiều nguồn, phân tích bằng ngôn ngữ SPL (Search Processing Language).

Giúp phát hiện bất thường, tạo cảnh báo, dashboard trực quan.

Mô hình hệ thống



HƯỚNG DẪN TẠO RULE WAF (OpenResty + Lua)

Mục tiêu: tạo, kiểm tra và tích hợp rule WAF trên OpenResty (nginx + Lua) để detect và block các tấn công web cơ bản (SQLi, XSS, ...). File này mô tả các bước thực hiện, nơi chạy lệnh và kỳ vọng output.

1. Yêu cầu trước khi bắt đầu

Môi trường (ví dụ của project):
- Kali Linux (máy attacker) — dùng để gửi request test (curl).
- Ubuntu (WAF + DVWA) — OpenResty/nginx chạy port 80, Apache/DVWA chạy nội bộ port 8080.
- Splunk Universal Forwarder (trên Ubuntu) và Splunk Indexer (10.10.10.28:8000).

Các file/đường dẫn quan trọng:
- Lua WAF chính: /usr/local/openresty/nginx/lua/waf.lua
- Nginx config: /usr/local/openresty/nginx/conf/nginx.conf
- WAF log: /usr/local/openresty/nginx/logs/waf.log
- Splunk forwarder: /opt/splunkforwarder/...

2. Bước 0

Luôn sao lưu file trước khi chỉnh sửa. Chạy trên Ubuntu (máy WAF):

```
sudo cp /usr/local/openresty/nginx/lua/waf.lua
/usr/local/openresty/nginx/lua/waf.lua.bak-$(date +%F_%H%M%S)
sudo cp /usr/local/openresty/nginx/conf/nginx.conf
/usr/local/openresty/nginx/conf/nginx.conf.bak-$(date +%F_%H%M%S)
```

Ý nghĩa: lưu bản gốc để dễ rollback khi có lỗi.

3. Cấu trúc rule & file waf.lua

Bạn có thể dùng file Lua sau (copy vào /usr/local/openresty/nginx/lua/waf.lua). File này chạy trên Ubuntu, được gọi bởi nginx `access_by_lua_file`.
Giải thích ngắn mỗi phần:
- WAF_MODE: 'DETECT' chỉ log, 'BLOCK' sẽ trả 403 và chặn.
- patterns: danh sách regex theo loại tấn công.
- waf_log: set biến để ghi vào access_log (waf_msg, waf_rule).
- matches_any: tìm pattern phù hợp (dùng ngx.re.find).
- Khi match: gọi waf_log, nếu BLOCK thì trả HTTP 403.

```
-- waf.lua (ví dụ đơn giản)
local WAF_MODE = "BLOCK"   -- "DETECT" or "BLOCK"
local dict = ngx.shared.waf_shdict

ngx.var.waf_msg = ngx.var.waf_msg or "-"
ngx.var.waf_rule = ngx.var.waf_rule or "-"
```

```
local whitelist = { ["127.0.0.1"]=true }

if whitelist[ngx.var.remote_addr] then
  return
end

local function waf_log(msg, rule)
  ngx.var.waf_msg = msg or "-"
  ngx.var.waf_rule = rule or "-"
  ngx.log(ngx.ERR, "[WAF] ", msg, " | rule=", rule or "-", " | ip=",
ngx.var.remote_addr or "-", " | uri=", ngx.var.request_uri or "-")
end

local function matches_any(subject, patterns)
  if not subject then return nil end
  local s = string.lower(subject)
  for _, p in ipairs(patterns) do
    if ngx.re.find(s, p, "jo") then
      return p
    end
  end
  return nil
end

local sqli_patterns = {"union\s+select","or\s+1=1","drop\s+table"}
local xss_patterns = {"<script","onerror\s*="}

local text = (ngx.var.request_uri or "") .. " " .. (ngx.var.args or "") .. " " ..
(ngx.var.http_user_agent or "")

local p = matches_any(text, sqli_patterns)
if p then
  waf_log("SQLi detected", p)
  if WAF_MODE == "BLOCK" then
    ngx.status = ngx.HTTP_FORBIDDEN
    ngx.say("403 Forbidden - blocked by WAF (SQLi)")
    return ngx.exit(ngx.HTTP_FORBIDDEN)
  end
  return
```

```
end

p = matches_any(text, xss_patterns)
if p then
  waf_log("XSS detected", p)
  if WAF_MODE == "BLOCK" then
    ngx.status = ngx.HTTP_FORBIDDEN
    ngx.say("403 Forbidden - blocked by WAF (XSS)")
    return ngx.exit(ngx.HTTP_FORBIDDEN)
  end
  return
end

return
```

4. Cấu hình nginx (log format + lua_shared_dict)

Chạy trên Ubuntu — chỉnh file /usr/local/openresty/nginx/conf/nginx.conf. Các dòng cần có:

```
http {
   lua_shared_dict waf_shdict 10m;
   log_format waflog '$remote_addr - $remote_user [$time_local] "$request" '
                     '$status $body_bytes_sent "$http_referer" "$http_user_agent" '
                     'waf_msg="$waf_msg" waf_rule="$waf_rule"';
   access_log /usr/local/openresty/nginx/logs/waf.log waflog;

   server {
      listen 80;
      server_name localhost;
      set $waf_msg "-";
      set $waf_rule "-";

      location /DVWA/ {
         access_by_lua_file /usr/local/openresty/nginx/lua/waf.lua;
         proxy_pass http://127.0.0.1:8080;
         proxy_set_header X-Real-IP $remote_addr;
         proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
      }
   }
}
```

Giải thích:

- `lua_shared_dict`: vùng lưu tạm cho Lua (auto-ban..).
- `log_format waflog ...`: định nghĩa file log có chứa waf_msg và waf_rule để Splunk dễ parse.
- `access_by_lua_file`: gọi script Lua cho location.

5. Kiểm tra và reload OpenResty

Chạy trên Ubuntu:
sudo /usr/local/openresty/nginx/sbin/nginx -t
sudo systemctl reload openresty
sudo systemctl status openresty --no-pager -l
Kỳ vọng:
- `nginx -t` báo syntax is ok.
- `systemctl status openresty` show Active: active (running).

6. Các bước test (mỗi lệnh chạy ở đâu)

A. Mở log realtime (Ubuntu) — Terminal A:
sudo tail -F /usr/local/openresty/nginx/logs/waf.log
B. Test từ Kali (Terminal B) — request bình thường:
curl -i http://10.10.10.30/DVWA/
Kỳ vọng: dòng log mới với waf_msg="-" (bình thường).
C. Test SQLi (Kali):
curl -i -G 'http://10.10.10.30/DVWA/vulnerabilities/sqli/' --data-urlencode "id=1 UNION SELECT"
Kỳ vọng: HTTP 403 (nếu BLOCK) và waf.log có waf_msg="SQLi detected" waf_rule contains pattern. Nếu DETECT mode thì request vẫn pass nhưng log có waf_msg.
D. Test XSS (Kali):
curl -i -G 'http://10.10.10.30/DVWA/vulnerabilities/xss_r/' --data-urlencode "name=<script>alert(1)</script>"

7. Tích hợp Splunk (Forwarder) — kiểm tra

Trên Ubuntu (forwarder): kiểm tra status và cấu hình:
sudo /opt/splunkforwarder/bin/splunk status
sudo sed -n '1,200p' /opt/splunkforwarder/etc/system/local/outputs.conf
sudo sed -n '1,200p' /opt/splunkforwarder/etc/system/local/inputs.conf
Kỳ vọng: forwarder đang chạy và inputs.conf chứa monitor cho /usr/local/openresty/nginx/logs/waf.log (index = waf_logs).
Trên Splunk Web (10.10.10.28:8000) search:
index="waf_logs" sourcetype="nginx:waf" | rex "waf_msg=\"(?<waf_msg>[^\"]+)\"\\s+waf_rule=\"(?<waf_rule>[^\"]+)\"" | table _time remote_addr request status waf_msg waf_rule | sort - _time

Kỳ vọng: thấy log normal và log attack; ví dụ có dòng wa f_msg="SQLi detected".

8. Chuyển giữa DETECT và BLOCK

Chỉnh WAF_MODE trong /usr/local/openresty/nginx/lua/waf.lua:
local WAF_MODE = "DETECT" -- chỉ log
local WAF_MODE = "BLOCK" -- chặn + log
Sau khi chỉnh: reload openresty (xem phần 5). Kiểm tra bằng test SQLi/XSS để thấy khác biệt.

9. Whitelist (bỏ qua IP để tránh chặn nhầm)

Ví dụ thêm whitelist trong waf.lua:

```
local whitelist = {
  ["127.0.0.1"] = true,
  ["10.10.10.100"] = true
}
if whitelist[ngx.var.remote_addr] then
  return
end
```

Giải nghĩa: nếu IP trong danh sách, script sẽ `return` sớm và không kiểm tra pattern.

10. Tuning, false-positive và troubleshooting

Một số lưu ý:
- Bắt đầu ở chế độ DETECT khi thêm rules mới để tránh chặn nhầm (false positive).
- Nếu thấy false positive: thu hẹp regex, ví dụ 'union\s+select' thay vì 'union'.
- Kiểm tra logs: /usr/local/openresty/nginx/logs/waf.log và /opt/splunkforwarder/var/log/splunk/splunkd.log
- Nếu forwarder không gửi: kiểm tra kết nối TCP tới indexer (nc -vz 10.10.10.28 9997).

Phụ lục A — Giải thích access_log format (waf.log)

Một dòng log mẫu và ý nghĩa từng trường:

```
10.10.10.2 - - [31/Oct/2025:19:11:20 +0700] "GET /DVWA/vulnerabilities/sqli/?id=1+UNION+SELECT HTTP/1.1" 403 49 "-" "curl/8.15.0" waf_msg="SQLi detected" waf_rule="union\x5Cs+select"
```

- remote_addr: 10.10.10.2 (IP client).
- time: [31/Oct/2025:19:11:20 +0700].
- request: method + path + HTTP version.
- status: HTTP response code (403 = blocked).
- waf_msg/waf_rule: result from Lua WAF.

Checklist nhanh trước demo

- OpenResty active and listening on port 80 (ss -tulnp).
- Apache (DVWA) running on port 8080 and reachable from OpenResty.
- Splunk Universal Forwarder running and configured to monitor waf.log.
- Splunk Index 'waf_logs' exists and search returns results.

Trên máy Ubuntu (WAF + Web) — cài Universal Forwarder

1. Tải và cài UF (DEB) — ví dụ cho Ubuntu

```
sudo dpkg -i splunkforwarder-10.0.1-c486717c322b-linux-amd64.deb
sudo dpkg -i splunkforwarder.deb
# start và chấp nhận license
sudo /opt/splunkforwarder/bin/splunk start --accept-license
# enable auto start
sudo /opt/splunkforwarder/bin/splunk enable boot-start
```

2. Cấu hình forward server (gửi tới Splunk indexer)

```
sudo /opt/splunkforwarder/bin/splunk add forward-server <SPLUNK_IP>:9997 -
admin:admin@123
```

Trên Splunk Server

1. Cài Splunk Enterprise

```
wget -O splunk-10.0.1-c486717c322b-linux-amd64.deb
"https://download.splunk.com/products/splunk/releases/10.0.1/linux/splunk-10.0.1-
c486717c322b-linux-amd64.deb"
sudo dpkg -i splunk-10.0.1-c486717c322b-linux-amd64.deb
sudo /opt/splunk/bin/splunk start --accept-license
```

user: admin
passwd: admin@123

2. Tạo index cho logs (để lưu dữ liệu gửi từ Forwarder)

Settings → Indexes → New Index

Tạo 2 index:

Index Name	Description
nginx	logs web (access/error)
modsec	logs ModSecurity

3. Bật "Receiving" trên port 9997 (để nhận log từ Universal Forwarder)

Vào Settings → Forwarding and receiving → Receive data
Nhấn New Receiving Port
Nhập port 9997, rồi Save

4. Tạo sourcetypes / CIM mapping

Loại log	Sourcetype
access log	access_combined_nginx
error log	nginx_error
modsecurity	modsecurity_audit

DVWA (Damn Vulnerable Web Application) là một công cụ học tập không thể thiếu cho những ai muốn tìm hiểu về bảo mật web. Khi cài đặt DVWA trên Linux, bạn sẽ được cung cấp môi trường an toàn cho việc học hỏi, rèn luyện các kỹ thuật tấn công và phòng thủ web. Trong bài viết này, mình sẽ hướng dẫn bạn từng bước cài đặt DVWA, từ chuẩn bị môi trường đến cấu hình để bắt đầu thực hành.

Bước 1: Tải xuống DVWA

```
cd /var/www/html
sudo git clone https://github.com/ethicalhack3r/DVWA
```

Bước 2: Cấu hình DVWA

```
chmod -R 777 DVWA/
cd dvwa/config
sudo mv config.inc.php.dist config.inc.php
sudo nano config.inc.php
```

Bước 3: Cài đặt MySQL

```
sudo apt install default-mysql-server
```

Bước 4: Cấu hình cơ sở dữ liệu MySQL

```
sudo service mysql start
systemctl status mysql
sudo mysql -u root -p
create user 'user'@'10.10.10.30' identified by 'pass';
grant all privileges on dvwa.* to 'user'@'10.10.10.30' identified by 'pass';
```

Bước 5: Cài đặt PHP

```
sudo apt update
sudo apt install php7.4 -y
sudo apt install php7.4-xxx
sudo apt install php7.4-{cli,json,imap,bcmath,bz2,intl,gd,mbstring,mysql,zip}
```

Bước 6: Cấu hình máy chủ Apache

```
cd /etc/php/7.4/apache2
ls /etc/php
sudo nano php.ini
service apache2 start
ifconfig
```

Bước 7: Truy cập DVWA trên trình duyệt

http://10.10.10.30:8080/DVWA

Đánh giá

Trong quá trình triển khai và thử nghiệm hệ thống “WAF Log Monitor”, nhóm nhận thấy mô hình hoạt động ổn định và đáp ứng được các mục tiêu đề ra. Hệ thống có khả năng phát hiện và ghi nhận các tấn công phổ biến như SQL Injection và Cross-

Site Scripting (XSS), đồng thời hiển thị log trực quan trên Splunk, giúp người quản trị dễ dàng theo dõi và phân tích hành vi tấn công.
Ưu điểm:
Hệ thống được triển khai dễ dàng, không yêu cầu phần cứng mạnh.
Chi phí thấp nhờ sử dụng các nền tảng mã nguồn mở như OpenResty, ModSecurity, và Splunk Free Edition.

Có khả năng tích hợp linh hoạt với Splunk, hỗ trợ thu thập và phân tích log tập trung.

Giúp sinh viên hiểu rõ quy trình bảo vệ web ở tầng ứng dụng và kỹ năng xử lý log thực tế.

Hạn chế:
Chưa có tính năng tự động chặn hoặc cô lập IP (auto-ban), do đó việc phản ứng với tấn công vẫn mang tính thủ công.
Hệ thống chưa được tối ưu về hiệu năng, có thể ảnh hưởng đến tốc độ xử lý khi có lượng truy cập lớn.
Chưa có cơ chế phân quyền hoặc dashboard bảo mật chuyên biệt cho người dùng cuối.

Hướng phát triển
Trong tương lai, nhóm mong muốn mở rộng và hoàn thiện mô hình WAF theo các hướng sau:

- Nâng cao bộ rule bằng cách cập nhật các mẫu tấn công mới từ OWASP CRS và cộng đồng bảo mật.

- Tích hợp cơ chế học máy (Machine Learning) để phát hiện hành vi bất thường tự động dựa trên mô hình thống kê hoặc hành vi truy cập.

- Bổ sung tính năng auto-ban IP hoặc tạm thời chặn truy cập khi phát hiện nhiều hành vi đáng ngờ.

- Xây dựng dashboard tùy chỉnh trên Splunk để hiển thị thống kê, biểu đồ, và cảnh báo theo thời gian thực.

- Nghiên cứu khả năng kết hợp với hệ thống SIEM hoặc IDS/IPS để tăng tính toàn diện cho mô hình phòng thủ.

KẾT LUẬN

Dự án "WAF Log Monitor" đã giúp nhóm sinh viên hiểu rõ hơn về cơ chế hoạt động của Web Application Firewall và quy trình phân tích log bảo mật trong môi trường thực tế. Thông qua việc triển khai Nginx (hoặc OpenResty) kết hợp với Splunk, nhóm đã xây dựng thành công một mô hình giám sát tấn công web cơ bản, có khả năng phát hiện và ghi nhận các yêu cầu đáng ngờ.

Kết quả thực nghiệm cho thấy hệ thống có thể phát hiện chính xác các tấn công SQLi và XSS, đồng thời ghi log chi tiết giúp người quản trị dễ dàng theo dõi và xử lý sự cố. Dự án không chỉ củng cố kiến thức về an toàn mạng và hệ thống mã nguồn mở, mà còn giúp sinh viên rèn luyện kỹ năng vận hành, phân tích và điều tra sự cố an ninh mạng — những năng lực quan trọng trong ngành An toàn thông tin.

Mặc dù còn một số hạn chế về tính năng tự động và khả năng mở rộng, dự án đã đạt được các mục tiêu học tập ban đầu và có tiềm năng phát triển thành một công cụ hỗ trợ giảng dạy hoặc nghiên cứu về bảo mật web trong tương lai.